



**RAJALAKSHMI ENGINEERING COLLEGE**

*Approved by AICTE | Affiliated to Anna University | Accredited by NAAC*

**MINI PROJECT REPORT ON**  
**TICKIT - A TASK MANAGEMENT MOBILE APP**

*Submitted by*

**MANISHAA G**

**2116230701176**

*in partial fulfilment of the award of the degree of*

**BACHELOR OF ENGINEERING**  
**IN**  
**COMPUTER SCIENCE AND ENGINEERING**

**RAJALAKSHMI ENGINEERING COLLEGE**

**ANNA UNIVERSITY, CHENNAI**

**MAY 2026**

## **BONAFIDE CERTIFICATE**

Certified that this Project titled “TickIt - A Task Management Mobile App” is the Bonafide work of “**MANISHAA G (2116230701176)**” who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Submitted for the End semester practical examination to be held on

---

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

## ABSTRACT

In an era driven by increasing digital workloads and the demand for seamless productivity across multiple devices, efficient task management and synchronization have become essential for modern users. TickIt is a modern cross-platform task management application developed using Kotlin Multiplatform and Compose Multiplatform, aimed at providing a unified and reliable productivity solution across Android and Desktop platforms. The application follows a local-first architecture that allows users to manage tasks efficiently even without internet connectivity.

***TickIt*** utilizes the Room database for secure local data storage and integrates seamlessly with the Google Tasks API for cloud synchronization. The system ensures that task operations such as creating, editing, deleting, and organizing tasks remain fully functional offline, while synchronization automatically occurs once network connectivity becomes available.

The application adopts a modular architecture consisting of separate modules for authentication, API communication, synchronization, business logic, and shared user interface components. Secure OAuth 2.0 authentication is implemented to provide safe access to Google services. The user interface is developed using Material Design 3 principles, offering a modern, responsive, and visually consistent experience across multiple platforms, including support for dark mode.

***TickIt*** demonstrates the effectiveness of modern cross-platform development techniques, shared UI implementation, and local-first synchronization strategies in building scalable productivity applications. The project showcases how a single shared codebase can deliver reliable performance, improved maintainability, and seamless user experience across devices while enhancing personal productivity and task organization.

## ACKNOWLEDGEMENT

Initially, we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavour to put forth this report. Our sincere thanks to our chairman **Mr. S. MEGANATHAN, B.E, F.I.E.**, our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.**, our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E, M.S.**, for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.**, our beloved principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. E.M MALATHY, M.E., Ph.D.**, Professor and Head of the Department of Computer Science and Engineering and our mentor **Mrs B.Deepa M.E., (Phd).**, Department of Computer Science and Engineering for his useful tips during our review to build our project.

# TABLE OF CONTENTS

| CHAPTER   | TITLE                                         | PAGE<br>NO |
|-----------|-----------------------------------------------|------------|
|           | ABSTRACT                                      | iii        |
|           | ACKNOWLEDGEMENT                               | iv         |
| <b>1.</b> | <b>INTRODUCTION</b>                           | <b>01</b>  |
|           | 1.1 GENERAL                                   | 01         |
|           | 1.2 SCOPE OF THE WORK                         | 02         |
|           | 1.3 AIM & OBJECTIVES OF THE<br>PROJECT        | 03         |
| <b>2.</b> | <b>LITERATURE SURVEY</b>                      | <b>04</b>  |
| <b>3.</b> | <b>SYSTEM DESIGN</b>                          | <b>06</b>  |
|           | 3.1 ARCHITECTURE DIAGRAM                      | 06         |
|           | 3.2 USE CASE DIAGRAM                          | 08         |
|           | 3.3 HARDWARE SPECIFICATIONS                   | 10         |
|           | 3.4 SOFTWARE SPECIFICATIONS                   | 11         |
| <b>4.</b> | <b>PROPOSED SYSTEM</b>                        | <b>13</b>  |
| <b>5.</b> | <b>MODULE DESCRIPTION</b>                     | <b>16</b>  |
| <b>6.</b> | <b>OUTPUT</b>                                 | <b>19</b>  |
| <b>7.</b> | <b>CONCLUSION AND FUTURE<br/>ENHANCEMENTS</b> | <b>20</b>  |
| <b>8.</b> | <b>REFERNCES</b>                              | <b>22</b>  |

# CHAPTER 1

## INTRODUCTION

### 1.1 GENERAL

**TickIt** is a modern cross-platform task management application developed using Kotlin Multiplatform and Compose Multiplatform. The application is designed with a local-first architecture that enables users to manage their daily tasks efficiently even without internet connectivity. TickIt synchronizes seamlessly with the Google Tasks API, ensuring that task data remains consistent across devices and platforms.

The application follows a modular and layered architecture that promotes code reusability, maintainability, and scalability. Most of the business logic, user interface components, and synchronization mechanisms are shared between Android and Desktop platforms through Kotlin Multiplatform technology. This approach reduces development effort while maintaining a consistent user experience across multiple platforms.

TickIt integrates several modern technologies such as Ktor for networking, Room database for local storage, Koin for dependency injection, and Material Design 3 for responsive and modern user interfaces. The application emphasizes offline functionality, fast performance, secure authentication using OAuth 2.0, and reliable synchronization with cloud services.

The app is structured into multiple modules including authentication, Google Tasks API integration, synchronization engine, shared UI components, and platform-specific launch modules. By combining modern mobile development practices with cross-platform capabilities, TickIt provides users with a seamless and efficient task management experience focused on productivity and simplicity.

## 1.2 SCOPE OF THE WORK

The scope of this project focuses on the design and development of TickIt, a modern cross-platform task management application that provides efficient task organization, synchronization, and offline accessibility across Android and Desktop platforms. The application is developed using Kotlin Multiplatform and Compose Multiplatform to maximize code sharing and maintain a consistent user experience across multiple devices.

The project primarily aims to implement a local-first architecture where all task-related data is stored locally using the Room database before being synchronized with the Google Tasks API. This enables users to create, edit, update, and delete tasks even when internet connectivity is unavailable. Once a network connection is restored, the synchronization engine reconciles local changes with cloud data to ensure consistency and reliability.

The scope also includes the implementation of secure OAuth 2.0 authentication, modular architecture design, task synchronization mechanisms, responsive user interface development, and platform-specific configurations for Android and Desktop environments. The application incorporates modern UI principles using Material Design 3 and supports features such as dark mode, responsive layouts, task list management, and real-time synchronization.

In addition, the project emphasizes maintainability and scalability through modular separation of concerns. Dedicated modules are created for API communication, authentication handling, local database management, dependency injection, and shared user interface components. This modular approach simplifies future enhancements and feature additions.

The project further focuses on performance optimization, offline reliability, and smooth cross-platform functionality while minimizing redundant code. Future scalability considerations include expanding support for additional platforms, improving synchronization reliability, adding notification systems, and integrating advanced productivity features.

## 1.3 AIM AND OBJECTIVES OF THE PROJECT

### **Aim:**

The primary aim of this project is to design and develop TickIt, a modern cross-platform task management application that enables users to efficiently organize, manage, and synchronize tasks across multiple devices using a local-first architecture. The application focuses on providing seamless offline functionality, reliable cloud synchronization, and a consistent user experience on both Android and Desktop platforms.

### **Objectives of the Project:**

- To develop a cross-platform task management application using Kotlin Multiplatform and Compose Multiplatform.
- To implement a local-first architecture using Room database for offline task storage and management.
- To integrate the Google Tasks API for seamless cloud synchronization.
- To provide secure user authentication using OAuth 2.0 mechanisms.
- To enable users to create, edit, delete, organize, and synchronize tasks efficiently.
- To design a modern and responsive user interface using Material Design 3 principles.
- To ensure smooth synchronization between local storage and cloud data even after network interruptions.
- To reduce code duplication by sharing business logic and UI components across Android and Desktop platforms.
- To implement modular architecture for better maintainability, scalability, and easier future enhancements.



## **CHAPTER 2**

### **LITERATURE SURVEY**

Recent studies on productivity and task management applications highlight the growing importance of cross-platform accessibility, offline synchronization, and user-centric design in modern mobile applications. Various research works and existing systems have contributed to the development concepts used in TickIt.

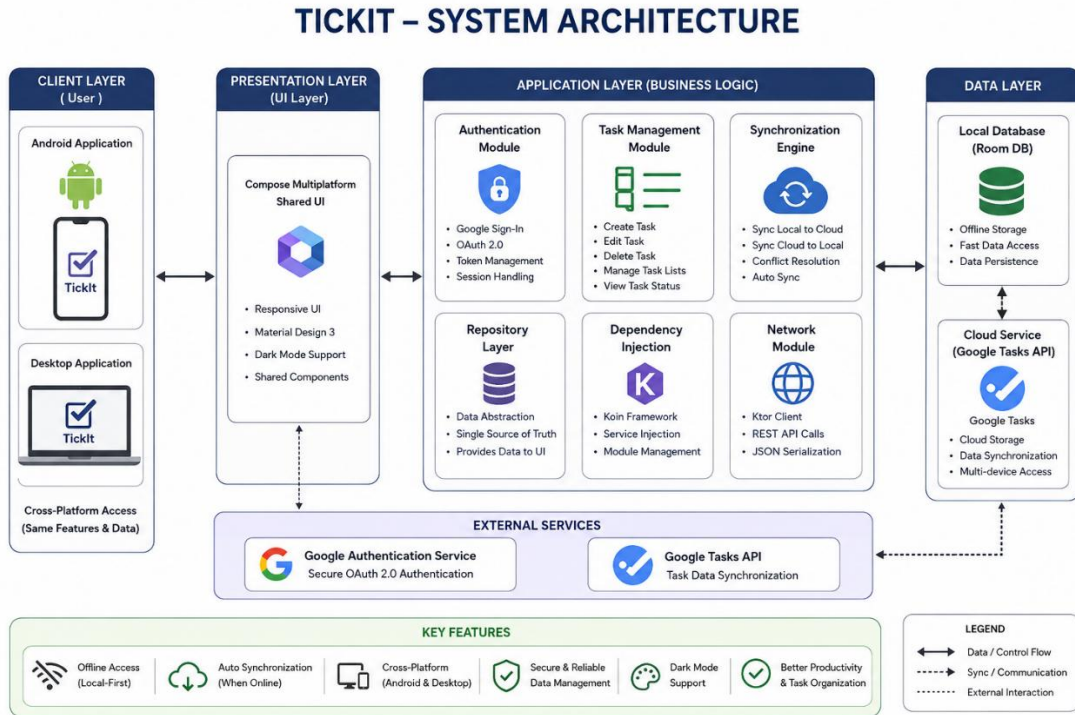
- In [1], a cross-platform productivity application developed using Kotlin Multiplatform demonstrates efficient code sharing between Android and Desktop platforms. The study highlights reduced development effort, improved maintainability, and consistent user experience across multiple devices. This research supports the cross-platform architecture used in TickIt.
- In [2], a local-first task management system using Room database and offline synchronization techniques is proposed. The system allows users to manage tasks without internet connectivity and later synchronize changes with cloud servers. Experimental results show improved reliability and uninterrupted task access during network failures. This research forms the basis for the local-first synchronization mechanism in TickIt.
- In [3], a cloud-based task synchronization framework integrated with the Google Tasks API is presented. The framework demonstrates secure task synchronization using OAuth 2.0 authentication and REST API communication. The study emphasizes secure authentication, efficient conflict handling, and reliable cloud updates, which directly support the Google Tasks integration in TickIt.
- In [4], a modern mobile application architecture using modular layered design is discussed. The research highlights how modularization improves scalability, code maintainability, independent testing, and easier feature expansion. This concept supports the modular project structure implemented in TickIt with separate modules for authentication, synchronization, API handling, and shared UI components.
- In [5], a user interface system developed using Jetpack Compose and Material Design 3 principles demonstrates improved UI responsiveness, simplified UI development, and enhanced user interaction. The study also highlights dark mode support and responsive layouts across devices. This research supports the Compose Multiplatform UI implementation used in TickIt.

- In [6], a dependency injection framework for Kotlin-based applications using Koin is discussed. The study explains how dependency injection improves modularity, simplifies service management, reduces code coupling, and enhances application scalability. This research supports the dependency management approach implemented in TickIt.
- In [7], a REST API communication framework using Ktor is presented for secure and efficient cloud communication in cross-platform applications. The framework demonstrates lightweight HTTP communication, JSON serialization, and asynchronous networking operations. This study supports the networking and cloud communication layer used in TickIt.
- In [8], a synchronization conflict resolution mechanism for offline-first applications is proposed. The system focuses on maintaining consistency between local databases and cloud servers during concurrent updates. The research highlights automatic synchronization recovery and reliable conflict handling, which support the synchronization engine implemented in TickIt.
- In [9], a modern productivity application with responsive dark mode support and adaptive layouts is introduced. The study demonstrates improved accessibility, reduced eye strain, and better user engagement through responsive UI design and theme customization. These concepts directly support the modern UI implementation and dark theme integration in TickIt.
- In [10], a cloud-integrated productivity management system with multi-device accessibility is discussed. The research highlights seamless synchronization across multiple devices, secure user authentication, and centralized task management. The study supports the cross-platform synchronization and cloud accessibility features implemented in TickIt.

# CHAPTER 3

## SYSTEM DESIGN

### 3.1 ARCHITECTURE DIAGRAM



**Fig 3.1.1 – Architecture Diagram**

The architecture of TickIt follows a modular layered design optimized for cross-platform development, offline functionality, and seamless cloud synchronization. The system is divided into multiple independent modules that handle authentication, API communication, local database operations, synchronization, and user interface management.

The application uses a local-first architecture where task data is initially stored in the local Room database before synchronization with the Google Tasks API. This approach ensures uninterrupted task management even during network failures. Once connectivity is restored, the synchronization engine updates both local and cloud data while resolving conflicts efficiently.

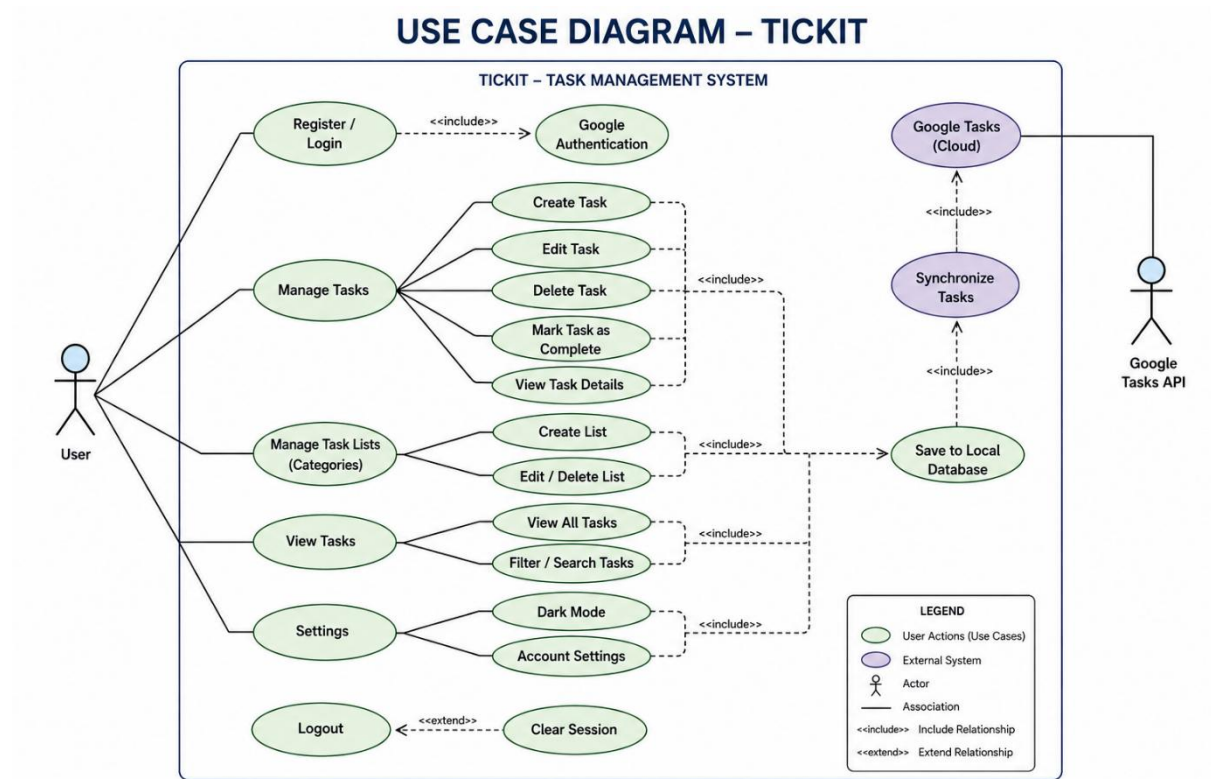
The core modules of the system include:

- **:oauth Module** – Handles OAuth 2.0 authentication and secure Google account login.
- **:tasks Module** – Provides API bindings and communication with the Google Tasks REST API using Ktor.
- **:tasks-core Module** – Acts as the central processing unit of the application containing Room database definitions, repositories, and synchronization logic.
- **:tasks-app-shared Module** – Contains the shared Compose Multiplatform UI components, themes, screens, and navigation logic.
- **:tasks-app-android Module** – Provides Android-specific configurations and application entry points.
- **:tasks-app-desktop Module** – Handles Desktop-specific window management and application launching.

The architecture also integrates Koin for dependency injection, enabling efficient service management and modular communication between components. Ktor handles networking operations, while Compose Multiplatform ensures a consistent user interface across Android and Desktop platforms.

The overall system architecture improves scalability, maintainability, code reusability, and synchronization reliability while providing users with a smooth and responsive task management experience.

### 3.2 USE CASE DIAGRAM



The Use Case Diagram of TickIt illustrates the interaction between the user and the major functionalities provided by the application. The system is designed to help users efficiently manage their daily tasks across multiple platforms while maintaining synchronization with the cloud.

The primary actor in the system is the **User**, who interacts with different modules of the application to perform task management operations.

## **Main Use Cases:**

- **User Authentication**

Allows users to securely sign in using their Google account through OAuth 2.0 authentication.

- **Create Tasks**

Enables users to create new tasks with titles, descriptions, due dates, and task lists.

- **Edit Tasks**

Allows modification of existing task details and updates both local and synchronized cloud data.

- **Delete Tasks**

Enables users to remove completed or unwanted tasks from the application.

- **Manage Task Lists**

Allows users to organize tasks into multiple task lists for better productivity management.

- **Offline Task Access**

Provides uninterrupted task access and editing even without internet connectivity using local database storage.

- **Synchronize Tasks**

Automatically synchronizes local task changes with the Google Tasks cloud service whenever internet connectivity is available.

- **View Task Status**

Allows users to monitor completed, pending, and updated tasks.

- **Dark Mode Support**

Enables users to switch between light and dark themes for better visual experience.

- **Cross-Platform Access**

Allows users to access the same task data seamlessly on Android and Desktop platforms.

### 3.3 HARDWARE SPECIFICATIONS

The TickIt application is designed to run efficiently on standard computing devices and Android smartphones without requiring specialized hardware. The cross-platform architecture ensures smooth performance across both Android and Desktop environments while maintaining reliable synchronization and offline functionality.

The following hardware specifications are recommended for optimal performance:

#### **Android Device Requirements**

- **Processor:**

Octa-core or Quad-core processor (Qualcomm Snapdragon 665 or higher / MediaTek equivalent)

- **RAM:**

Minimum 4 GB RAM (Recommended: 6 GB for smoother multitasking)

- **Storage:**

Minimum 64 GB internal storage

- **Display:**

HD or Full HD touchscreen display

- **Operating System:**

Android 8.0 (Oreo) or higher

- **Network:**

Internet connection required for Google Tasks synchronization

#### **Desktop Requirements:**

- **Processor:**

Intel Core i3 or higher

- **RAM:**

Minimum 4 GB RAM

- **Storage:**

Minimum 2 GB free disk space

- **Operating System:**

Windows 10/11, macOS, or Linux

### 3.4 SOFTWARE SPECIFICATIONS

The TickIt application is developed using modern cross-platform development technologies to provide efficient task management, seamless synchronization, and responsive user experience across Android and Desktop platforms. The software stack is designed with a modular architecture that improves scalability, maintainability, and performance.

#### **Frontend Framework**

- **Compose Multiplatform**  
Used for developing shared user interface components across Android and Desktop platforms.
- **Jetpack Compose**  
Used for building modern, responsive, and declarative UI components

#### **Programming Language**

- **Kotlin**  
Used as the primary programming language for both frontend and backend logic.

#### **Backend & Networking**

- **Ktor**  
Used for HTTP communication, REST API integration, and JSON serialization.
- **Google Tasks API**  
Used for cloud-based task synchronization and task management services.

#### **Database & Local Storage**

- **Room**  
Used for local database management and offline task storage.
- **Local-first synchronization mechanism**  
Ensures offline accessibility and automatic synchronization once internet connectivity is restored.

#### **Authentication**

- **OAuth 2.0 Authentication**  
Used for secure Google account login and API authorization



## **Dependency Injection**

- Koin

Used for dependency management and modular service injection.

## **UI Design & Styling**

- Material Design 3

Used for creating modern, responsive, and user-friendly interfaces.

- Dark Theme Support

Provides improved user experience and accessibility in low-light environments.

## **Image Loading**

- Coil

Used for loading user profile images and graphical assets efficiently.

## **Development Tools**

- Android Studio

Used for Android development and project management.

- IntelliJ IDEA

Used for Kotlin Multiplatform and Desktop development.

## **Version Control**

- GitHub

Used for source code management, version control, and collaboration.

These software technologies collectively ensure reliable synchronization, efficient offline storage, modern UI rendering, secure authentication, and smooth cross-platform performance in TickIt.

## **CHAPTER 4**

### **MODULE DESCRIPTION**

The TickIt application is developed using a modular architecture where each module is responsible for a specific functionality. This modular design improves scalability, maintainability, code reusability, and synchronization efficiency across Android and Desktop platforms.

Below is the description of the major modules in the system:

#### **1. Authentication Module (:oauth)**

The Authentication Module handles secure user authentication and authorization using OAuth 2.0 protocols. It enables users to sign in with their Google account and securely access Google Tasks services.

##### **Features:**

- Secure Google account login
- OAuth 2.0 token generation and validation
- Session management
- API authorization handling
- Secure access to cloud synchronization services

This module ensures safe communication between the application and Google services while protecting user credentials and task data.

#### **2. Google Tasks API Module (:tasks)**

The Google Tasks API Module manages communication with the Google Tasks REST API using Ktor.

##### **Features:**

- Fetching task lists from cloud storage
- Creating and updating cloud tasks
- Deleting synchronized tasks
- JSON serialization and deserialization
- REST API communication handling

This module acts as the bridge between the application and cloud-based task services.

### **3.Tasks Core Module (:tasks-core)**

The Tasks Core Module is the central processing unit of the application. It contains the business logic, local database management, repositories, and synchronization engine.

#### **Features:**

- Local database operations using Room
- Repository pattern implementation
- Offline task storage
- Synchronization conflict handling
- Task processing and state management

This module ensures efficient task management and reliable offline functionality.

### **4. Shared UI Module (:tasks-app-shared)**

The Shared UI Module contains almost all user interface components developed using Compose Multiplatform.

#### **Features:**

- Shared screens and navigation
- Reusable UI components
- Material Design 3 implementation
- Responsive layouts
- Dark mode support
- Cross-platform UI consistency

This module minimizes duplicate UI development across Android and Desktop platforms.

### **5. Android Application Module (:tasks-app-android)**

The Android module serves as the Android-specific entry point of the application.

#### **Features:**

- AndroidManifest configuration
- Android lifecycle integration
- Mobile-specific initialization
- Platform-specific settings

This module launches the shared UI within the Android environment.

## **6.Desktop Application Module (:tasks-app-desktop)**

The Desktop module manages Desktop-specific configurations and application launching.

### **Features:**

- Desktop window management
- Desktop lifecycle handling
- Cross-platform execution support
- Platform-specific initialization

This module allows TickIt to function smoothly on desktop operating systems.

## **7. Dependency Injection Module**

This module uses Koin for managing dependencies across the application.

### **Features:**

- Service injection
- Module management
- Reduced code coupling
- Simplified component communication

Dependency injection improves scalability and maintainability of the system.

## **8. Image and Asset Management Module**

This module handles image loading and graphical asset management using Coil.

### **Features:**

- Profile image loading
- Efficient image caching
- Asset optimization
- Smooth image rendering

This module enhances the visual experience and application performance.

## **CHAPTER 5**

### **PROPOSED SYSTEM**

The proposed system for TickIt is designed to provide a modern, efficient, and cross-platform task management solution that supports both offline accessibility and cloud synchronization. The system integrates local storage mechanisms with cloud-based services to ensure uninterrupted productivity and seamless task management across Android and Desktop platforms.

The application follows a local-first architecture where all user actions such as task creation, editing, deletion, and updates are initially stored in the local Room database. This enables users to continue working even without internet connectivity. Once network access becomes available, the synchronization engine automatically updates the cloud database through the Google Tasks API.

The proposed system is divided into multiple independent modules to improve scalability, maintainability, and performance. Each module performs a specific functionality while interacting efficiently with other components.

#### **Key Components of the Proposed System**

##### **1. Authentication Module**

- Handles secure user login using OAuth 2.0 authentication.
- Provides secure access to Google Tasks services.
- Manages user session and API authorization tokens.

##### **2. Task Management Module**

- Allows users to create, edit, delete, and organize tasks.
- Supports multiple task lists for better organization.
- Provides task completion tracking and status updates.

##### **3. Local Database Module**

- Uses Room database for offline task storage.
- Maintains fast local access to task information.
- Ensures data persistence even without internet connectivity.

#### **4. Synchronization Engine**

- Synchronizes local data with Google Tasks cloud storage.
- Detects task updates, additions, and deletions automatically.
- Resolves synchronization conflicts between local and cloud data.

#### **5. Shared UI Module**

- Implements shared user interface components using Compose Multiplatform.
- Ensures consistent UI experience across Android and Desktop platforms.
- Supports responsive layouts and dark mode.

#### **6. Networking Module**

- Uses Ktor for API communication.
- Handles HTTP requests, JSON serialization, and cloud connectivity.
- Ensures secure and efficient data transmission.

#### **7. Dependency Injection Module**

- Uses Koin for service management.
- Improves modularity and simplifies component integration.

The proposed system provides a unified productivity solution with reliable offline functionality, secure cloud synchronization, responsive user interfaces, and scalable architecture. The modular design also supports future enhancements such as notifications, reminders, additional platform support, and advanced productivity features.

## **5.1 KEY BENEFITS OF THE PROPOSED SYSTEM**

### **Cross-Platform Compatibility**

- Provides a consistent user experience across Android and Desktop platforms.

### **Offline Functionality**

- Allows users to manage tasks without internet connectivity using local database storage.

### **Reliable Synchronization**

- Automatically synchronizes local and cloud task data using Google Tasks API.

### **Modern User Interface**

- Uses Material Design 3 and Compose Multiplatform for responsive and attractive UI design.

### **Secure Authentication**

- Ensures safe access through OAuth 2.0 authentication.

### **Modular Architecture**

- Simplifies future enhancements, maintenance, and scalability.

### **Improved Productivity**

- Enables efficient task organization, tracking, and management within a single application.

## CHAPTER-6 OUTPUT



Fig 1: LOGIN PAGE



Fig 2: TASK MANAGER

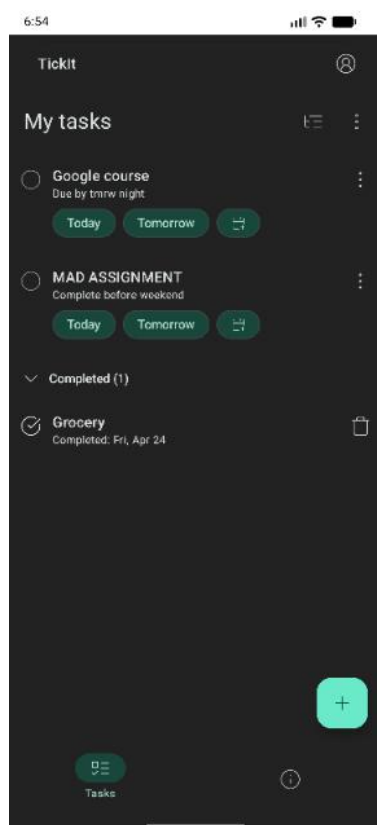


Fig 3: MY TASKS

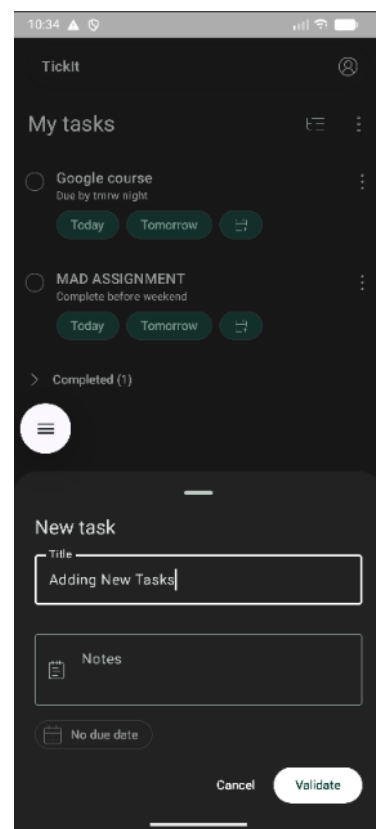


Fig 4: ADDING NEW TASKS



## CHAPTER 7

### CONCLUSION AND FUTURE ENHANCEMENTS

#### Conclusion

TickIt is a modern cross-platform task management application developed using Kotlin Multiplatform and Compose Multiplatform with a strong focus on offline functionality, cloud synchronization, and user productivity. The application successfully integrates local database storage with the Google Tasks API to provide reliable task management across Android and Desktop platforms.

The project demonstrates the effectiveness of modular architecture, shared UI development, and local-first synchronization strategies in modern mobile and desktop application development. By utilizing technologies such as Room database, Ktor, Koin, and Material Design 3, TickIt delivers a responsive, scalable, and user-friendly productivity solution.

The application enables users to create, organize, update, and synchronize tasks seamlessly while maintaining functionality even without internet connectivity. The shared codebase significantly reduces development complexity and ensures consistency across platforms. Overall, TickIt provides an efficient and modern productivity tool that enhances task organization, accessibility, and user experience.

#### Future Enhancements

##### 1. Push Notifications and Reminders

- Implement task reminders and notification alerts for upcoming deadlines and important tasks.

##### 2. Additional Platform Support

- Extend application support to platforms such as iOS and Web using Kotlin Multiplatform capabilities.

##### 3. Advanced Task Features

- Add features such as task priorities, labels, recurring tasks, and collaborative task sharing.

##### 4. AI-Based Productivity Suggestions

- Integrate AI-based recommendations for task prioritization and productivity optimization.

## **5.Enhanced Synchronization**

- Improve synchronization reliability with advanced conflict resolution mechanisms and background syncing.

- 6. Calendar Integration
  - Integrate external calendar services such as Google Calendar for better scheduling and planning.

## **7. Analytics Dashboard**

- Provide productivity analytics and task completion statistics to help users monitor progress.

- 8. Cloud Backup and Export
  - Implement backup and export functionality for secure long-term task storage and migration.

## **9. Widget and Quick Access Features**

- Add home screen widgets and quick task creation shortcuts for faster accessibility.

## **10. Team Collaboration Features**

- Introduce shared task lists and collaborative workspace functionality for teams and organizations.

## **CHAPTER 8**

### **REFERENCES**

- [1] J. JetBrains, “Kotlin Multiplatform for Cross-Platform Application Development,” JetBrains Documentation, 2024.
- [2] Google Developers, “Google Tasks API Documentation,” Google Cloud Documentation, 2024.
- [3] A. Vasquez and R. Martin, “Offline-First Mobile Application Architecture Using Local Databases and Cloud Synchronization,” *International Journal of Mobile Computing*, vol. 12, no. 3, pp. 45–53, 2022.
- [4] S. Kumar and P. Sharma, “Modern Cross-Platform UI Development Using Compose Multiplatform,” *Journal of Software Engineering and Applications*, vol. 15, no. 4, pp. 101–110, 2023.
- [5] Ktor Documentation, “Ktor Client for REST API Communication,” JetBrains, 2024.
- [6] Room Documentation, “Room Persistence Library Guide,” Android Developers Documentation, 2024.
- [7] Koin Documentation, “Dependency Injection with Koin,” InsertKoin.io, 2024.
- [8] Jetpack Compose Documentation, “Material Design 3 and Compose UI Development,” Android Developers Documentation, 2024.
- [9] R. Singh and M. Verma, “Secure OAuth 2.0 Authentication for Mobile Applications,” *International Journal of Cyber Security*, vol. 8, no. 2, pp. 88–96, 2021.
- [10] GitHub Documentation, “Version Control and Collaborative Software Development,” GitHub Docs, 2024.